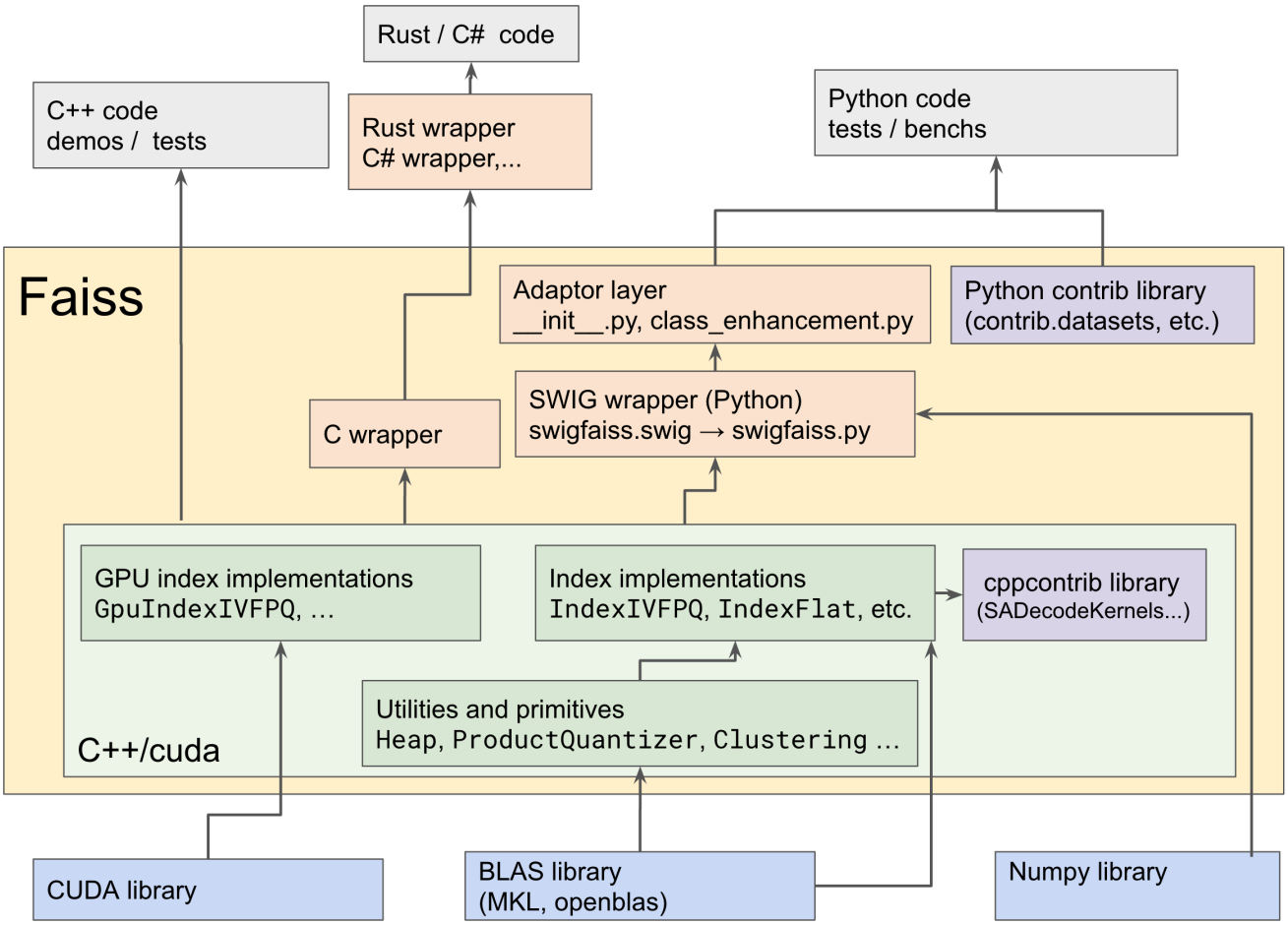
Faiss code structure

Jump to bottom

Matthijs Douze edited this page on Mar 31 · 30 revisions

Overall architecture

Faiss has a layered architecture that uses several computer languages.



Compilation

A Makefile compiles Faiss and its Python interface. It depends on Makefile variables that set various flags (BLAS library, optimization, etc.), and that are set in makefile.inc. See INSTALL on how to set the proper flags.

Dependencies

The only hard dependency of Faiss is BLAS/Lapack. It was developed with Intel MKL but any implementation that offers the API with fortran calling conventions should work. Many implementations are transitioning from 32- to 64-bit integers, so the type of integer that should be used must be specified at compile time with the `-DFINTEGER=int` or `-DFINTEGER=long` flag.

CPU Faiss C++ coding conventions

The CPU part of Faiss is intended to be easy to wrap in scripting languages. The GPU part of Faiss was written separately and follows different conventions (see below).

No public/private

All objects are C++ structs, there is no notion of public/private fields. All fields can be accessed directly. This means there are not safeguards that can be implemented via getters and setters. On the other hand, it makes accessing internals of the indexes from external code much easier since the code of Faiss does not need to be changed nor does the library need to be recompiled.

Object ownership (C++)

Faiss classes are intended to be as simple as possible so that the default copy constructors work as expected and the destructor is empty. There are a few exceptions, where an object A maintains a pointer to another object B. In this case, there is a boolean flag `own_fields` in A that indicates whether B should be deleted when A is deleted. The flag is set to false (not owned) by constructors. This will result in a memory leak if the reference to the B is lost in the calling code. You can set it to true if you want object A to destroy object B when it is destroyed. Library function like `index_factory`, `load_index` and `clone_index`, that construct A and B set it to true.

Class	field
IndexIVF	quantizer
IndexPreTransform	chain
IndexIDMap	index
IndexRefineFlat	base_index

For example, you can construct an IndexIVFPQ in C++ with

```
faiss::Index *index;
if (...) { // on output of this block, index is the only object that needs to be tracked
   faiss::IndexFlatL2 * coarseQuantizer = new  faiss::IndexFlatL2(targetDim);
   // you need to use a pointer here
   faiss::IndexIVFPQ *index1 = new faiss::IndexIVFPQ(
        coarseQuantizer, targetDim, numCentroids, numQuantizers, 8);
   index1->own_fields = true; // coarse quantizer is deallocated by index destructor
   index1->nprobe = 5;
   index = index1;
}
```

There is no shared_ptr implemented, mainly because it is hard to make it work with the ref-counts managed by Lua and Python.

Object ownership (Python)

SWIG generates a Python objects to wraps all C++ objects accessed in Python. When the object is constructed in Python (eg. via a constructor or `read_index`), the C++ object is deleted when the Python object is deleted. For composite objects constructed in Python, like

```
def make_index():
    coarseQuantizer = faiss.IndexFlatL2(targetDim)
    index = faiss.IndexIVFPQ(
        coarseQuantizer, targetDim, numCentroids, numQuantizers, 8)
```

```
        index.nprobe = 5
        return index
```

The C++ object's `own_fields` is set to `false` (the default), so Python needs to keep track of the object ownership. For this, constructors that take object arguments also add the object to `referenced_objects`, a Python list added dynamically to the python object. To give constructors this property, they are passed to `add_ref_in_constructor`.

A simple test to see if the object ownership works is to compare:

```
b = B()
a = a(b)
a.do_something()
```

and

```
a = a(B())
a.do_something()
```

If the second version crashes (segfault) and the first does not, it means that something is messed up with object ownership.

Coding conventions

Faiss is written using C++20. Avoid using templates in user-exposed code. Virtual functions are ok.

Please take into consideration the LEAR coding guidelines (also here).

Class names are CamlCase, methods and fields are lower_case_with_underscores (like in Python). All classes have a constructor without parameters that initializes parameters to some reproducible default, to make writing the I/O code easier. Indentation is 4 spaces for C++ and Python. This is enforced via clang-format, please run it before submitting a PR.

Do not use `long`, `long long`, `char` (except for strings) as they are not portable. Use explicitly sized types like `uint8_t`, `int64_t` for any integer type in an array.

Write simple code. C++ is a very complex language with a large standard library, and a functionality can be expressed in many different ways. Please choose the simplest way, that a beginner can understand.
A few guidelines:

- avoid abstractions unless you actually need them now.
- avoid most of the algorithm library (like `copy_if`, `count_if`, etc.), prefer explicit loops
- avoid code that does nothing (like getters / setters, functions that are not used, etc)

Optimization

Optimized code is often bloated, repetitive and hard to read.

Before optimizing, write a readable non-optimized version first. This serves:

- to understand how the algorithm works
- as a reference implementation to compare the optimized results
- as a baseline for new optimization sessions (for new hardware)

Preferably make the non-optimized version accessible from the caller, even if the optimized version covers all the cases. This means avoid putting the non-optimized version in comments, but instead having a flag that enables the non-optimized case.

Avoiding code bloat

This is an important objective in Faiss because the library is already very large. If you find yourself copy-pasting code often, it's a bad sign. Bloated code is harder to maintain because changes must be applied to all instances of the same code. Counting coder performance by the number of LOC they write is a heresy.

There are several ways to avoid code bloat:

- classical function refactoring
- templates (or preprocessor macros). Unfortunately, this makes code hard to read and compile errors even worse.
- being picky about belongs inside the library. Faiss is about vector search, and there are many functionalities that do not need to be in the core library. That's what `contrib` is for. There is an ongoing effort to make Faiss more modular so that writing specialized code is feasible outside of the library without performance penalty.
- scripting languages (build scripts and tests) *can* often include macros and loops, please use them to avoid code bloat there too.
- carefully control your coding assistant and instruct it to write compact code. Claude is pretty good at being terse when asked but by default it will write verbose code, especially for tests.

GPU Faiss coding conventions

The GPU Faiss index objects inherit from the CPU versions and provide some (but not all) of the same interface. For various reasons, not all of the CPU interface functions could be implemented, but the main ones are implemented. Getters and setters are preferred as there are often state implications on the GPU for changes (and certain errors need to be enforced). As a result, there is some degree of mismatch between GPU and CPU APIs, which we may attempt to clean up in the future.

`.cu` and `.cuh` files potentially contain CUDA and are meant for compilation (and inclusion) by `nvcc`. `.h` and `.cpp` files are meant for compilation by the host compiler. Linking is best done with `nvcc`, as it "knows" which cuda libraries to link in.

Tests

All new code should be tested. Tests are implemented in `tests/`. The tests should preferably be in Python because it is more compact and because they exercise the whole call stack (C++ tests do not test the wrapping layer).

Please fragment the tests, for example:

```
# BAD!
class Test1(unittest.TestCase):

    def test_all(self):
        for k in 1, 2, 3:
            self.assersEqual(faiss.add_10(k), k + 10)
```

```
# GOOD!
class Test1(unittest.TestCase):

    def do_test(self, k):   # will not be run by unittest because it does not start with "test_"
        self.assersEqual(faiss.add_10(k), k + 10)
```

```
    def test_1(self):
        self.do_test(1)

    def test_2(self):
        self.do_test(2)

    def test_3(self):
        self.do_test(3)
```

This makes it easier to narrow down what case is failing from the test run. Of course in this case the code of do_test could be put directly in the test_X functions, but real tests are often longer.

Inclusion in external projects

By convention, Faiss .h files are referred to as `<faiss/...>` (with angular brackets). For example, use

```
#include <faiss/IndexIVFPQ.h>
#include <faiss/gpu/GpuIndexFlat.h>
```

SIMD interfacing

Single Instruction, Multiple Data (SIMD) is used heavily in Faiss to speed up many types of operations. This includes AVX2, AVX512 (various flavors) for x86_64 CPUs and NEON, SVE for ARM CPUs. SIMD code that is run on a machine that does not support it will crash with `SIGILL`, therefore it is important to select the right implementation for the current machine.

As of 2026-02-24 Faiss is transitioning from a monolithic SIMD model to a dynamic dispatch model. New code should be written with dynamic dispatch in mind. We describe both below, taking AVX2 as an example. For more details, refer to this documentation.

Monolithic SIMD

The whole library is compiled with the `-mavx2` flag, that enables the `__AVX2__` macro. Code that is explicitly optimized for AVX2 should be guarded with `#ifdef __AVX2__`. The compiler may also choose to use AVX2 registers or instructions where it sees fit.

This is easy for the developer, since AVX2 instructions can be inserted wherever s/he wants. However, at deployment time, it means that Faiss needs to be compiled several times with several SIMD settings (currently AVX2, AVX512 and AVX512_SPR for x86_64). In C++, the client code needs to be linked to the right SIMD version. In Python, the python layer selects the right `swigfaiss_XXX.so` version of the library.

Dynamic dispatch SIMD

SIMD code is isolated in a specific file suffixed with `avx2`, see for example distances_avx2.cpp. Only these files get compiled with `-mavx2`, so that the compiler does not accidentally issue AVX2 instructions on machines that don't support it.

Orchestration between SIMD levels relies on an enum with all the SIMD levels. When a function, eg. `fvec_L2sqr` is specialized for a certain level, it is tagged with a template `fvec_L2sqr<SIMDLevel::AVX2>`. The version that is not tagged with a template dispatches dynamically to the currently selected SIMD level (given `SIMDConfig::level`). To avoid the overhead of dispatching this level inside tight loops, a code block can be specialized for a certain level with `with_simd_level`, see eg. here.

The code gets the macro `COMPILE_SIMD_AVX2` so that at compile time it can determine that `fvec_L2sqr<SIMDLevel::AVX2>` will be available (otherwise there would be linking errors).

At runtime, the SIMD level is set to the highest available level, but this can be changed via `SIMDConfig::set_level` or via the environment variable `FAISS_SIMD_LEVEL`.

This makes deployment much easier. However, it is more complex for developers because SIMD code needs to be isolated in specific files.

Transition period

During the transition period, Faiss can be compiled both as a monolithic implementation and dynamic dispatch mode. The dynamic dispatch is enabled via a CMake flag.

Note that during this period, in dynamic dispatch, part of the library is compiled without SIMD optimization (until the correspoinding SIMD code is moved to isolated files).

C++ code wrapping

The wrapping of C++ is done in two levels.

SWIG level

The C++ code can be called in Python thanks to SWIG. SWIG parses the Faiss header files and generates classes in Python for all the C++ classes it finds. The header files to be parsed are specified in swigfaiss.swig, which is currently monolithic (but we are working on it!).

The SWIG module is called `swigfaiss` in Python, this is the low-lever wrapper. The functions and class methods can be called transparently from Python. For example the C++ class `faiss::IndexFlat` appears as `swigfaiss.IndexFlat` in Python with all its fields and methods.

The Faiss module

The `faiss` module is an additional level of wrapping above `swigfaiss`. The main entry point is faiss/python/**init**.py.

Its role is to:

- loading the appropriate swigfaiss version (GPU or not, SIMD optimization level) and re-export it entirely
- re-define methods that handle pointers (see below). This is done in class_wrappers.py, see the `replacement_XXX` functions.
- track implicit references between C++ object. See the `add_ref_in_constructor` functions.

Handling of C++ pointers

The only tricky part is when the C++ classes expect pointers to arrays. SWIG does not automatically handle these. Therefore, the Faiss SWIG wrapper `faiss.py` adds the following:

- a function to extract a SWIG pointer from the data in an array: this is `swig_ptr(numpy.array)`. The functions are type-checked and they check if the array is C-compact, but there is no protection against NULL/nil/None and no verification of the array size, so is may cause the usual C/C++ crashes or memory corruptions.
- for the most common functions (`Index::train`, `Index::add`, `Index::search`, etc.) a specific wrapper was written to accept arrays in the scripting language directly, and verify their sizes. The original function is moved aside (eg. `train -> train_c`) and replaced with a wrapper function that does the proper type conversion.
- to convert a `std::vector<> v` to a numpy array, use `faiss.vector_to_array(v)`, this copies the data to numpy (works for vectors of scalar elements).

- to copy the content of a numpy array to a `std::vector<> v`, use `faiss.copy_array_to_vector(a, v)`. The vector will be resized and will contain only the data of `a`.
- to create a numpy array that references a `float* x` pointer, use `rev_swig_ptr(x, 125)`. The numpy array will be 125 elements long. The data is not copied and size is not checked. To use this with `std::vector<float>`'s use the `.data()` method of the vector to get the pointer to its data.

Pages 56

Home

Tutorial

Installing Faiss

Getting started

Faster search

Lower memory footprint

Running on GPUs

Basics

MetricType and distances

Faiss building blocks: clustering, PCA, quantization

Guidelines to choose an index

Faiss indexes

Basic indexes

Binary indexes

Composite indexes

Pre- and post-processing

The index factory

Index IO, cloning and hyper parameter tuning

Special operations on indexes

Additive quantizers

CPU Faiss + Intel SVS - Overview

CPU Faiss + Intel SVS - Usage

GPU Faiss

GPU overview

GPU versus CPU

Sample: GPU k-means

GPU Faiss + NVIDIA cuVS - Overview

GPU Faiss + NVIDIA cuVS - Usage

Advanced topics

Faiss code structure

Threads and asynchronous calls

Inverted list objects and scanners

Indexes that do not fit in RAM

Vector codecs

Brute force search without an index

Fast accumulation of PQ and AQ codes (FastScan)

Implementation notes

Setting search parameters for one query

How to make Faiss run faster

Typical use cases and benchmarks

Low level benchmarks

Indexing 1M vectors

Indexing 1G vectors

Indexing 1T vectors

Vector codec benchmarks

Binary hashing index benchmark

Hybrid CPU/GPU and multiple GPUs

Case studies

Additional information

FAQ

Python/C++ code snippets

Troubleshooting

Related projects

Clone this wiki locally

```
https://github.com/facebookresearch/faiss.wiki.git
```